

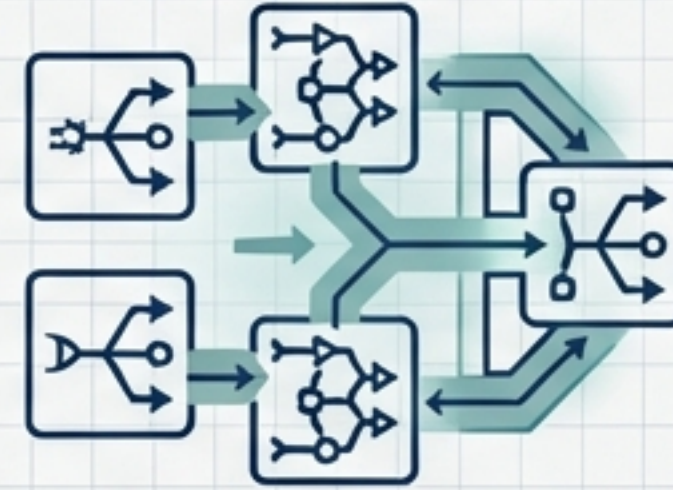
Sifat-Sifat Bahasa Reguler: Empat Pilar Analisis

Peta Jalan Membedah, Memanipulasi, dan Mengoptimasi Finite Automata



Pumping Lemma

Membuktikan kontradiksi dan batas komputasi absolut dari bahasa non-reguler.

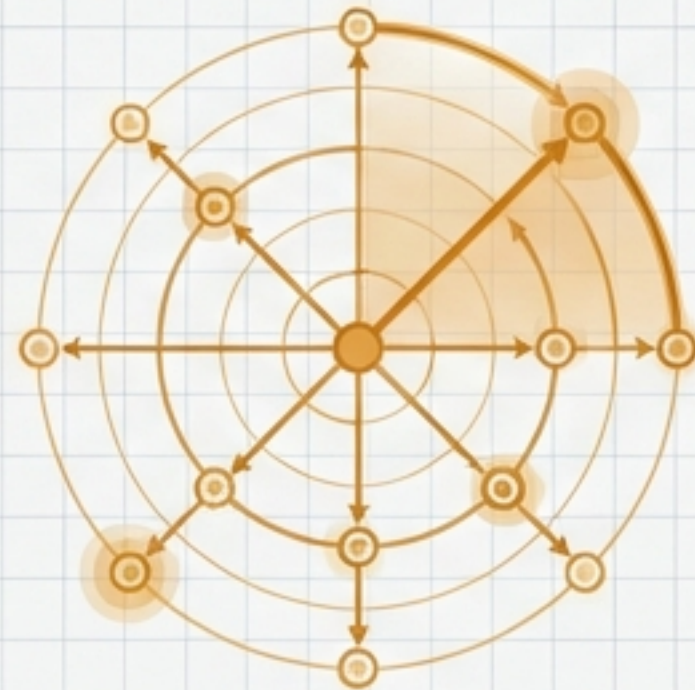


Closure Properties

Rekayasa dan perakitan automaton baru melalui serangkaian operasi matematis.

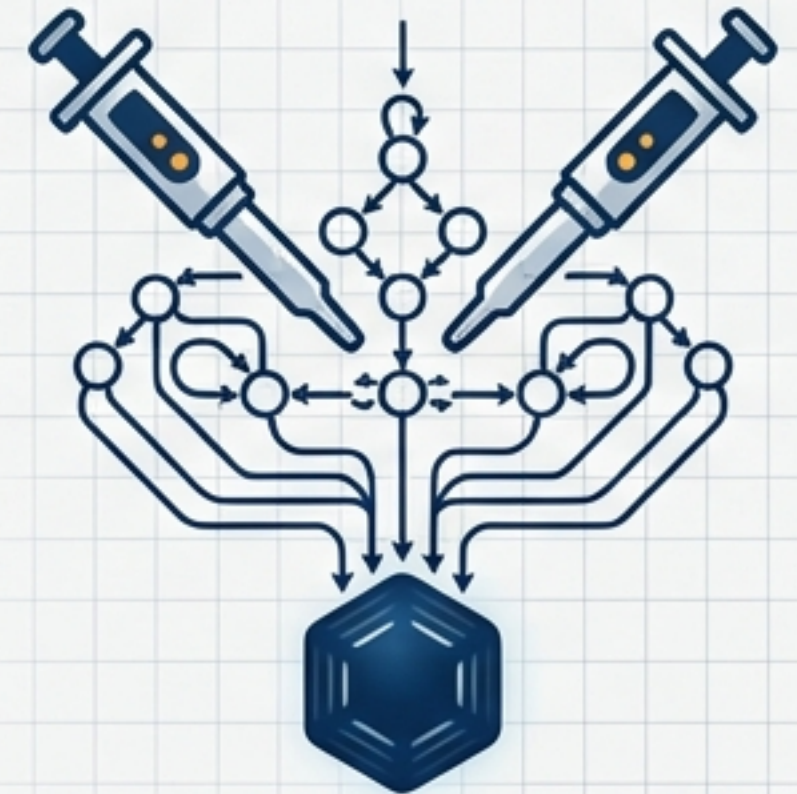
Decision Properties

Analisis komputasional algoritmik untuk keterjangkauan, ekuivalensi, dan keanggotaan.



Minimization Techniques

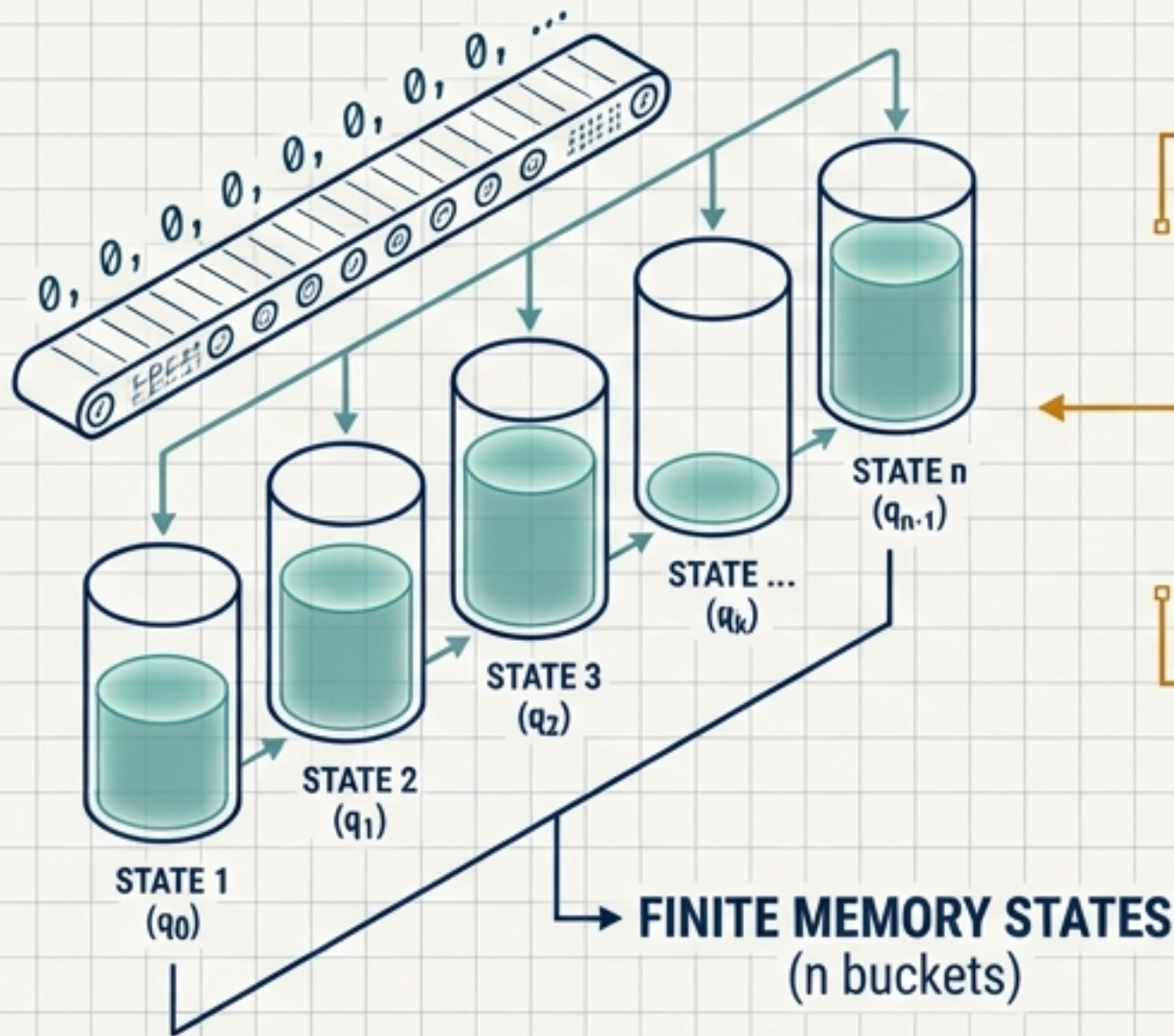
Reduksi state berlebih untuk memahat struktur DFA yang paling efisien dan unik.



KETERBATASAN MEMORI PADA FINITE AUTOMATA

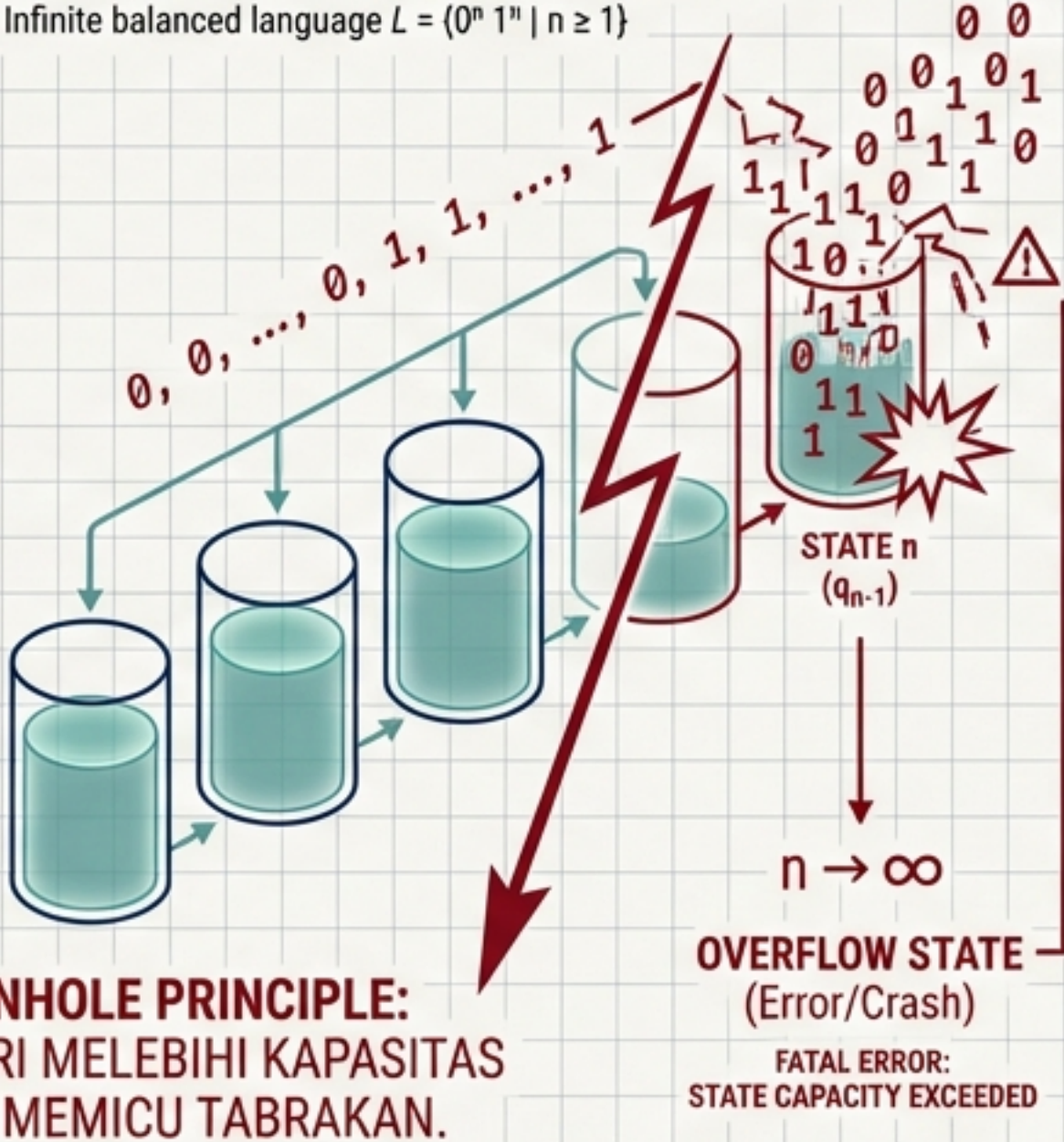
Mengapa Ada Bahasa yang Tidak Bisa Dikenali Komputer Paling Sederhana?

THE MACHINE: FINITE STATE ARCHITECTURE (DFA)



THE CRASH: PIGEONHOLE PRINCIPLE OVERFLOW

Infinite balanced language $L = \{0^n 1^n \mid n \geq 1\}$



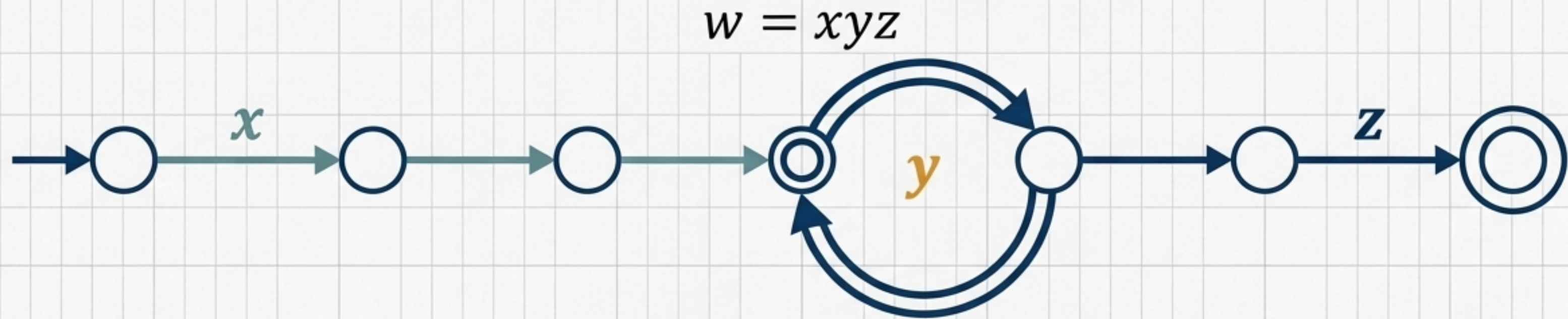
PIGEONHOLE PRINCIPLE:
MEMORI MELEBIHI KAPASITAS
STATE MEMICU TABRAKAN.

SCHEMATIC ID: FA-LIM-001

ENGINEER: AUTO-GEN

TEOREMA PUMPING LEMMA: ANATOMI LOOPING

Setiap String Panjang Pasti Terjebak dalam Perulangan State



Jika L adalah bahasa reguler, ada konstanta n di mana string w dengan $|w| \geq n$ mematuhi $w = xyz$:

- | | |
|------------------------------------|--|
| 1. $y \neq \epsilon$ | (Segmen loop tidak boleh kosong / harus eksis) |
| 2. $ xy \leq n$ | (Loop pasti terjadi di dalam radius n state pertama akibat batas memori) |
| 3. $\forall k \geq 0, xy^kz \in L$ | (Memompa putaran y sebanyak apa pun akan tetap bermuara pada jalur z yang valid) |

PEMBUKTIAN SEBAGAI ADVERSARIAL GAME

Mematahkan Kebohongan dengan Logika Permainan Dua Pihak

LAWAN (Universal $\forall$)

Memilih konstanta n sembarang.

Memecah w menjadi x, y, z sambil mematuhi syarat $|xy| \leq n$ dan $y \neq \epsilon$.

KITA (Eksistensial $\exists$)

Secara taktis memilih string $w \in L$ di mana $|w| \geq n$.

Mengeksekusi jebakan!
Pilih jumlah pompa k sedemikian rupa sehingga hasil $xy^kz \notin L$.

CHECKMATE.

Kontradiksi terbukti. Mesin lawan hancur.
Bahasa tersebut dipastikan BUKAN bahasa reguler.

SCHEMATIC ID
PROOF-GAME-001

ENGINEER
AUTO-GEN

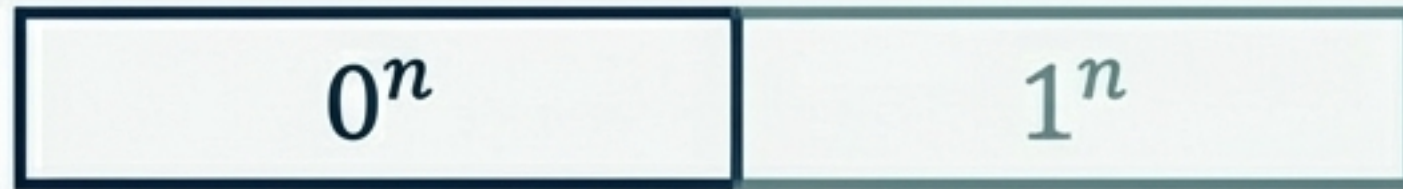
KASUS UJI PUMPING LEMMA: MEMBONGKAR DUA BAHASA

Eksekusi Taktik Permainan pada Skenario Matematis Nyata

KASUS 1: KESEIMBANGAN SEMU

Bahasa: Jumlah '0' sama dengan '1' (L_{eq})

Taktik String: Pilih $w = 0^n 1^n$



x, y jatuh di zona 0 karena $|xy| \leq n$

Eksekusi $k = 2$:

Memompa y menghasilkan $0^{n+|y|} 1^n$.
Keseimbangan jumlah angka rusak.

BUKAN REGULER

KASUS 2: FAKTOR PRIMA

Bahasa: 1^p , di mana p adalah bilangan prima (L_{pr})

Taktik String: Pilih $w = 1^p$ dengan $p \geq n+2$.
Misal panjang $|y| = m$.

$$\begin{aligned}\text{Panjang baru} &= |xz| + (p-m)|y| \\ \text{Panjang baru} &= (1+m)(p-m)\end{aligned}$$

Hasil pemfaktoran menunjukkan panjang string sekarang adalah bilangan komposit (hasil kali dua bilangan > 1), bukan lagi prima.

BUKAN REGULER

Sifat Ketertutupan (Closure Properties)

Memanipulasi Bahasa Reguler Tanpa Merusak Sifat Dasarnya

Bahasa Reguler L

Bahasa Reguler M

Operasi
Matematis

Dijamin 100% Reguler

✓ Union

✓ Intersection

✓ Complement

✓ Difference

✓ Reversal

✓ $h(x)$ Homomorphism

Closure properties membuktikan bahwa dengan mengoperasikan komponen bahasa reguler, kita secara otomatis merakit Finite Automata (FA) baru yang valid. Ini adalah fondasi rekayasa Regular Expressions.

SCHEMATIC ID
CLOSURE-PROP-001

ENGINEER
AUTO-GEN

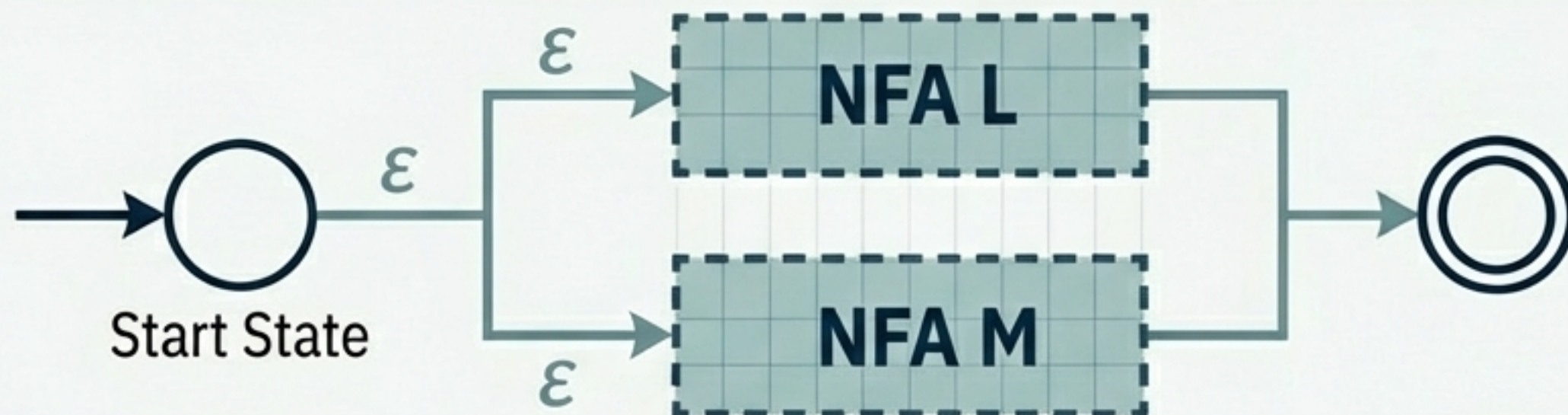
Operasi Ketertutupan Dasar: Union & Complement

Menggabungkan Cabang Sirkuit dan Membalik Logika Penerimaan

Union ($L \cup M$)

Ekspresi:

$$L(E) \cup L(F) = L(E + F)$$



Complement ($\bar{L}$)

Definisi:

Semua himpunan string dalam alfabet yang TIDAK ada di L.



Intersection via Konstruksi Produk

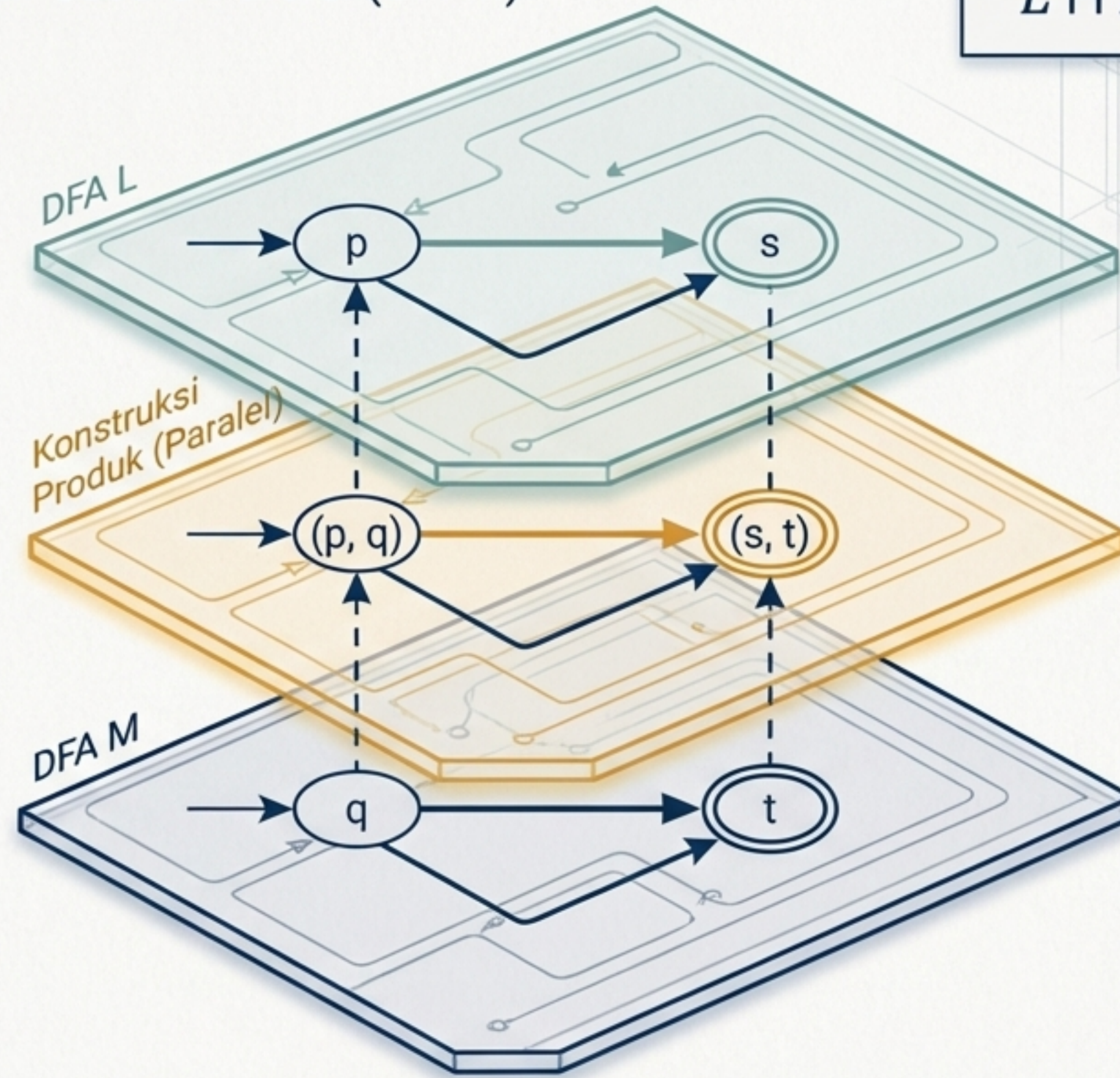
Simulasi Dua Mesin Automata Berjalan Secara Paralel ($L \cap M$)

Alternatif via Hukum DeMorgan:

$$L \cap M = (\bar{L} \cup \bar{M})$$

Syarat Penerimaan:

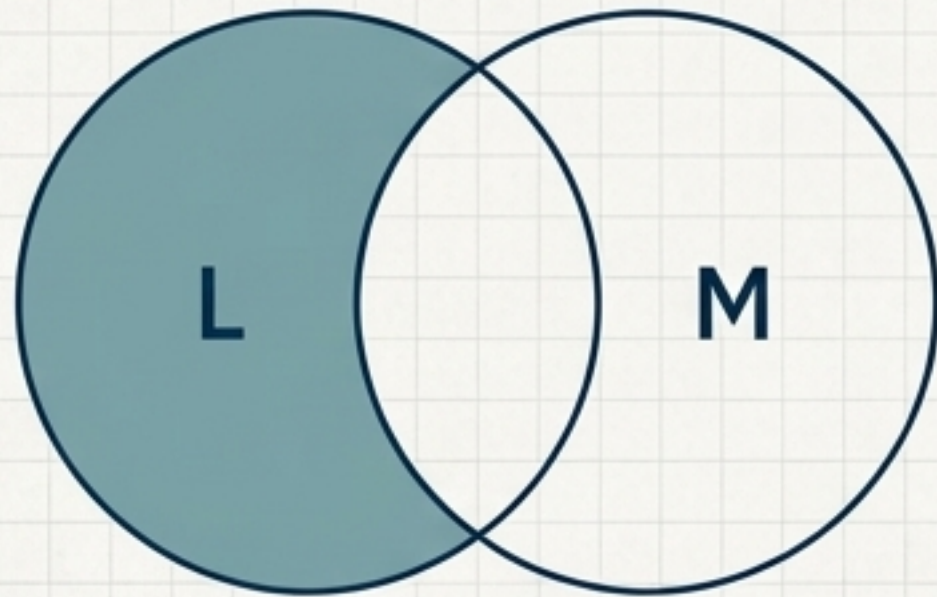
State gabungan (s, t) berstatus Accepting HANYA JIKA kedua state tunggal (s dan t) merupakan Accepting State di bidang asalnya masing-masing.



Difference & Reversal (Pembalikan Arah)

Memotong Himpunan dan Memutar Waktu Komputasi

Difference ($L \setminus M$)



Eksekusi Struktural:

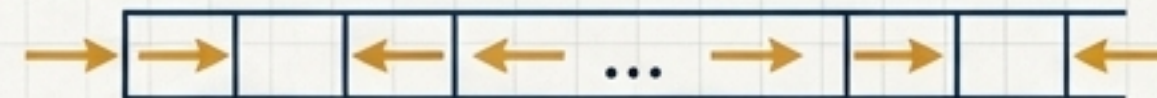
- Dapat dilakukan secara instan tanpa algoritma baru menggunakan operasi dasar yang telah dibuktikan:

$$L \setminus M = L \cap \overline{M}$$

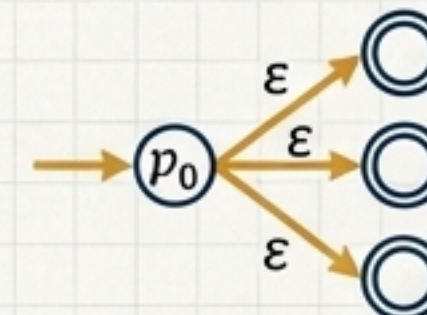
Reversal (L^R)

$w^R \in L^R$ adalah bayangan cermin dari $w \in L$.

Blueprint 3 Langkah



Start



1. Balik Panah: Putar balik arah SEMUA lintasan transisi.

2. Tukar Identitas: Start State lama diubah menjadi satu-satunya Accepting State.

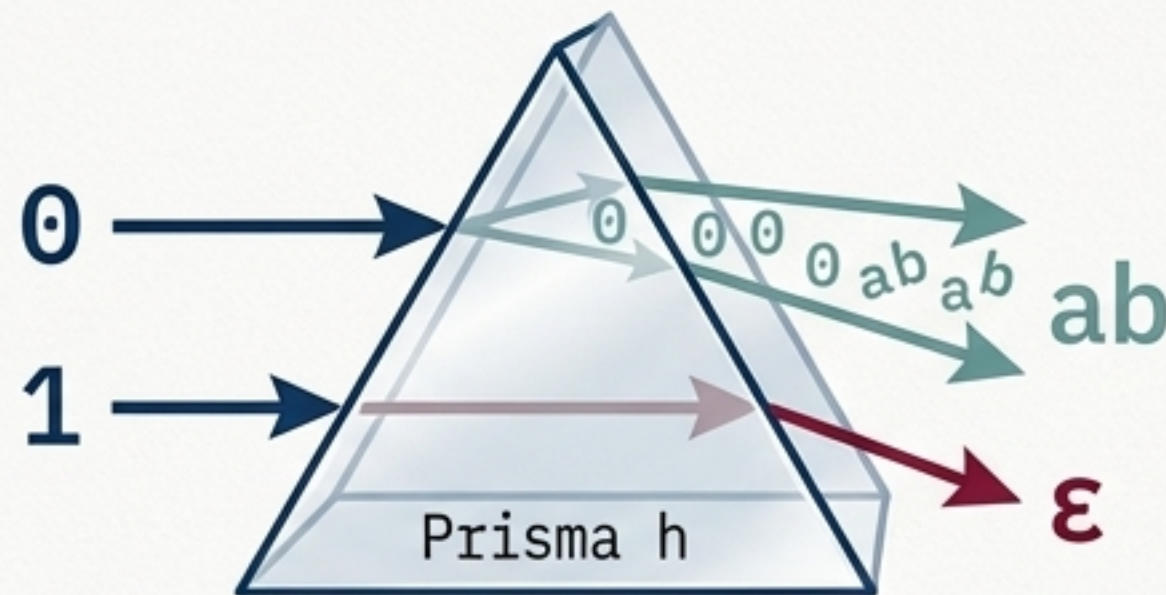
3. Inisiasi Baru: Buat Start State baru (p_0) dengan transisi- ϵ menyebar ke seluruh mantan Accepting State.

Homomorphism & Inverse Homomorphism

Transformasi Simbol Menggunakan Lensa Pemetaan Fungsional

Homomorphism $h(L)$ - Substitusi Proaktif

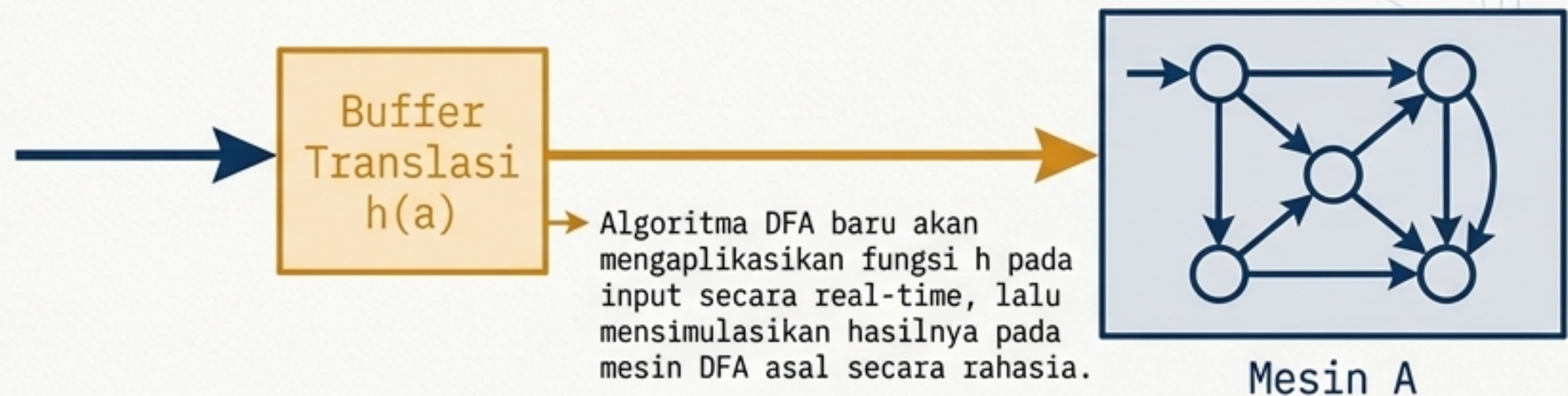
Mengganti satu simbol dengan rentetan string langsung pada Regular Expression.



Contoh Eksekusi:
 $10^*1 \rightarrow \epsilon(ab)^*\epsilon \rightarrow (ab)^*$

Inverse Homomorphism $h^{-1}(L)$ - Lacak Mundur

Mencari himpunan string asal yang bila dipetakan akan masuk dan diterima oleh L .



Kompleksitas Konversi Representasi

Mengukur Biaya Komputasional di Balik Transformasi Mesin

Jenis Konversi	Kompleksitas Waktu	Status Beban	
RE → ε-NFA (Membangun Tree)	$O(n)$	Ringan	⚙️
Eliminasi ε-transitions	$O(n^3)$	Polinomial Sedang	⚖️
NFA / ε-NFA → DFA	$O(2^n)$	Eksplosif (Bottleneck)	💣
DFA → NFA (Menambah subset)	$O(n)$	Trivial	↺↻
FA → Regex	$O(n^3 4^n)$	Sangat Eksplosif	🚨

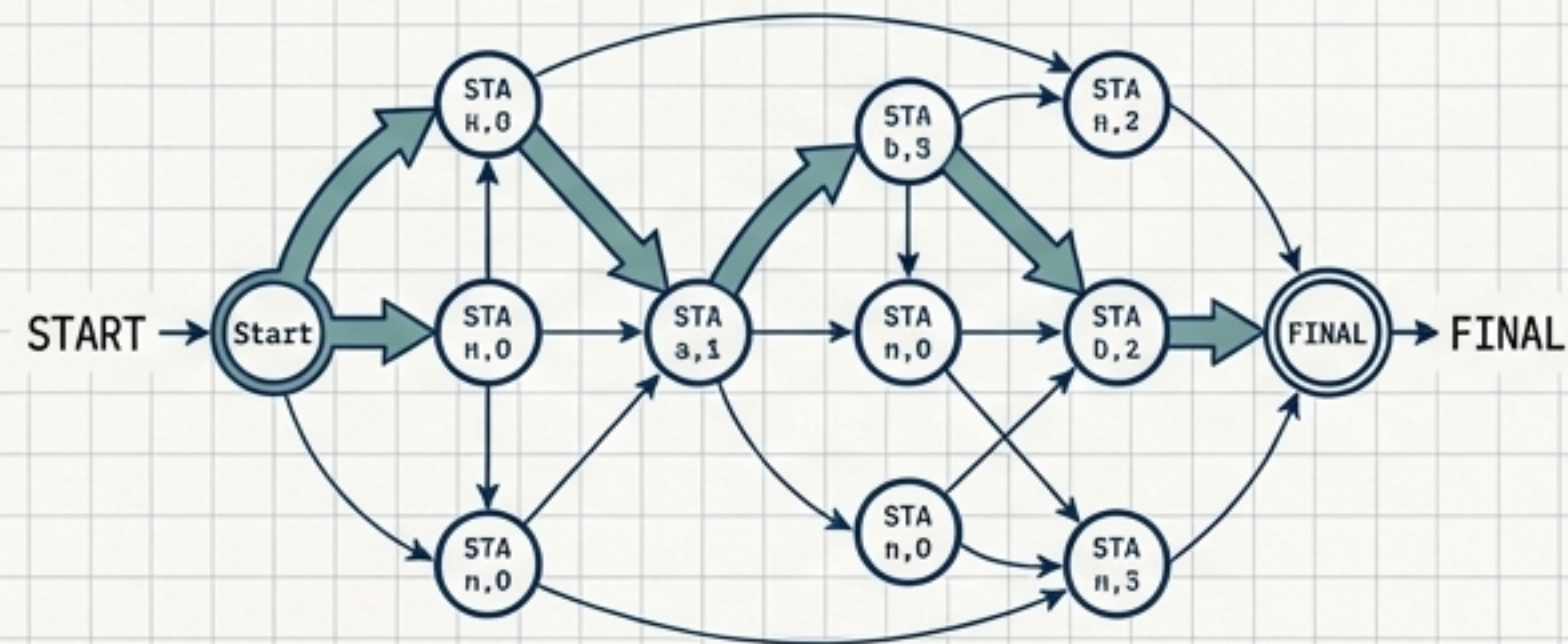
Peringatan Konstruksi Subsystem: Mengonversi mesin non-deterministik menjadi mesin deterministik memicu ledakan ruang state melalui algoritma subset construction. **Waspada biaya eksponensial ini!**

Decision Properties: Emptiness & Membership

Navigasi Graf vs Simulasi String pada Arsitektur Mesin

Testing Emptiness ($L = \emptyset$?)

Apakah mesin memiliki jalan dari Start menuju Final?

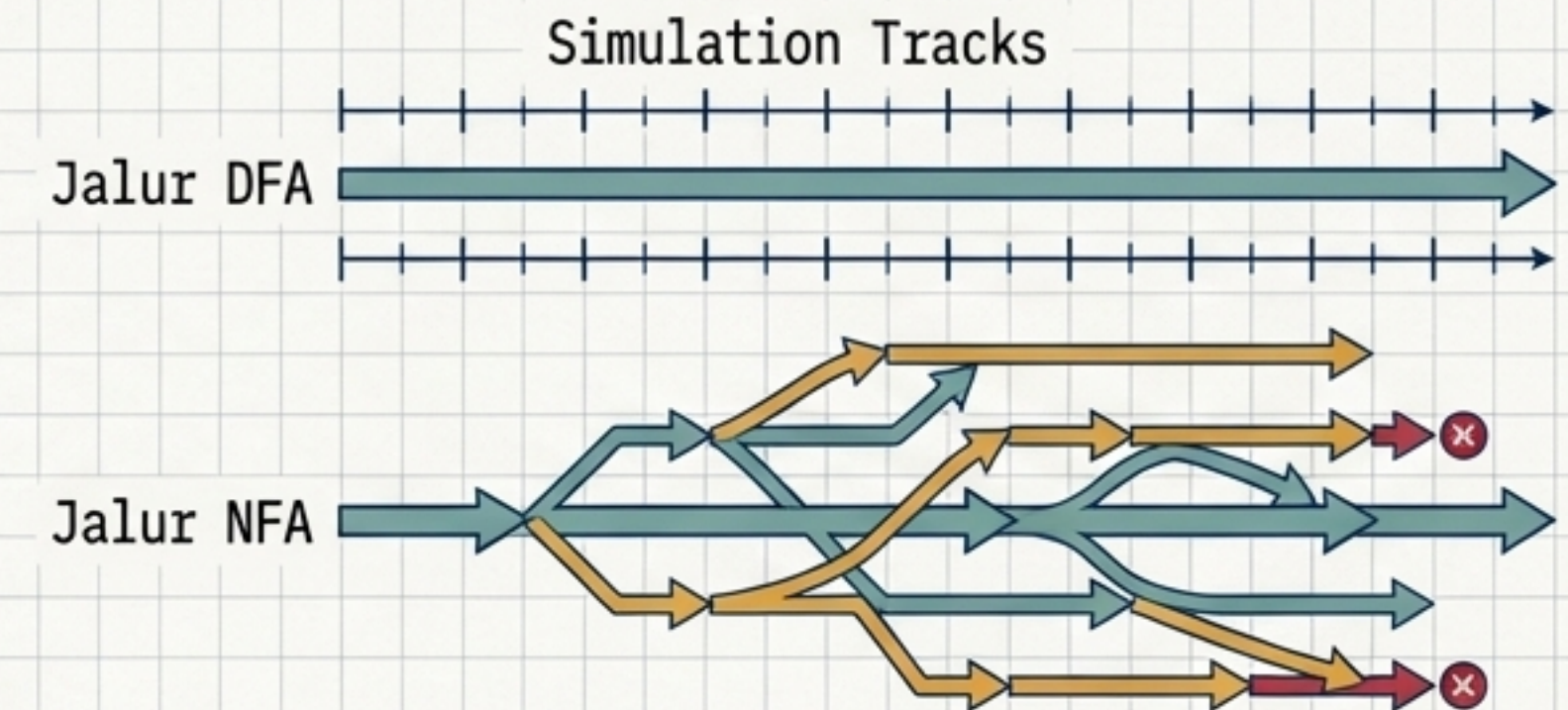


Metode: Algoritma Graph Reachability

Kompleksitas: Sangat efisien $\rightarrow O(n^2)$

Testing Membership ($w \in L$?)

Apakah string w secara presisi diterima oleh mesin?



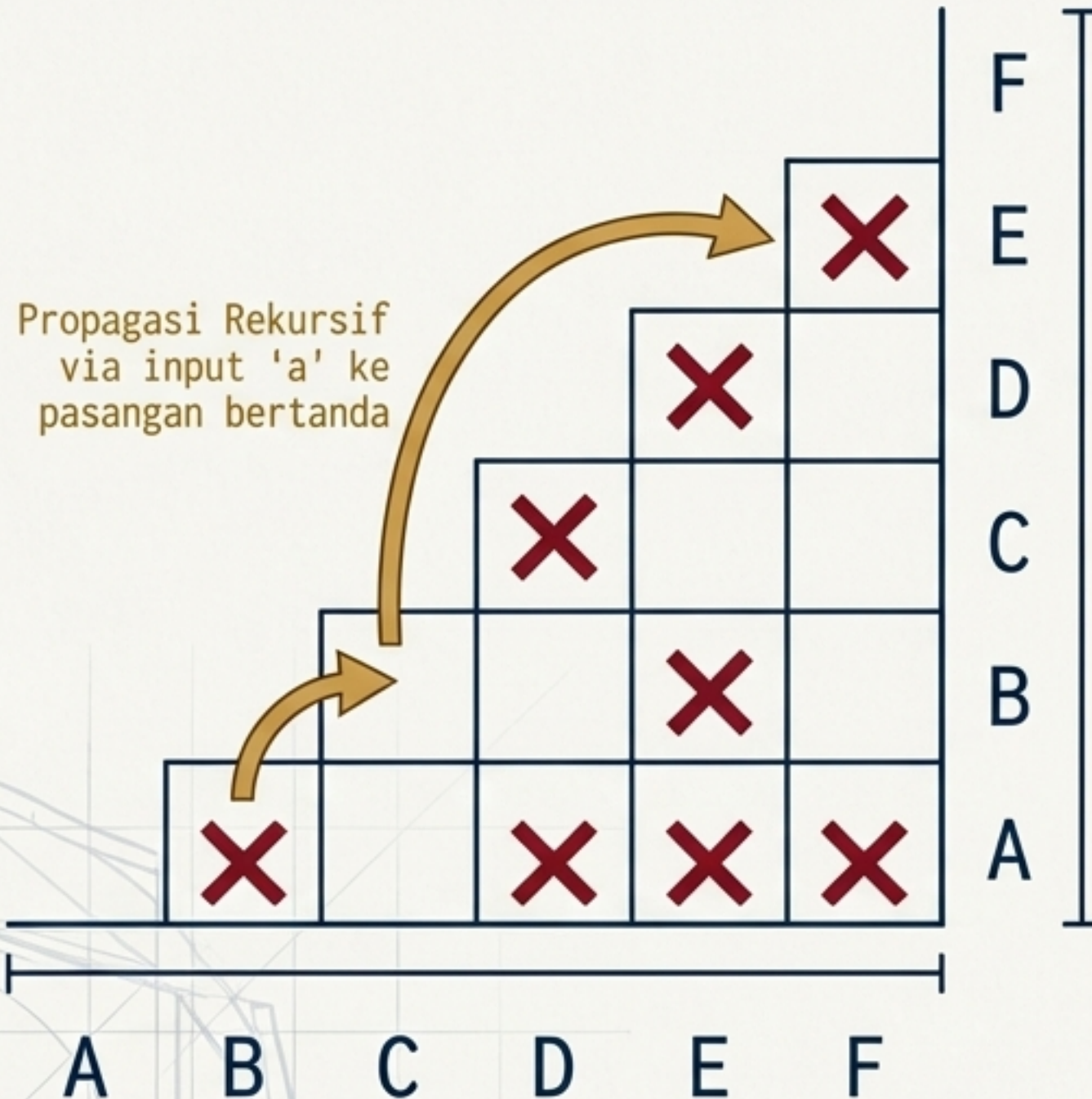
DFA: Simulasi lurus instan $\rightarrow$ Waktu $O(|w|)$

NFA: Menyimpan banyak kemungkinan state tiap langkah $\rightarrow$ Waktu $O(|w| \times s^2)$ hingga $O(|w| \times s^3)$

DFA menang telak secara absolut dalam simulasi runtime, menjustifikasi biaya mahal saat pembangunannya.

Algoritma Table-Filling

Melacak dan Memisahkan Pasangan State yang Distinguishable



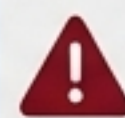
1 Langkah 1: Basis (Inisiasi Perbedaan)

- Berikan tanda silang (X) pada matriks untuk semua pasangan kombinasi (Accepting State, Non-Accepting State).
- Logika: Jika satu menolak dan satu menerima, mereka secara mutlak berbeda.



2 Langkah 2: Induksi (Efek Domino)

- Jika dari state asal merambat membaca input 'a' dan mendarat pada pasangan state yang SUDAH memiliki tanda (X), maka kotak state asal TERINFEKSI dan wajib diberi tanda (X).
- Proses ini menjalar rekursif berulang hingga tidak ada tanda silang yang bisa ditambahkan.



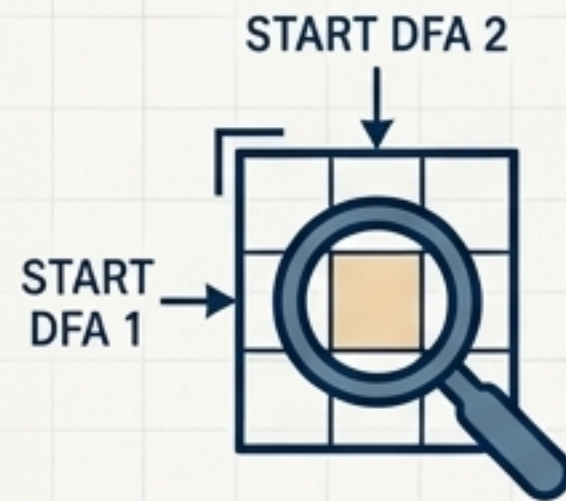
Konklusi: Semua kotak matriks yang TIDAK ditandai (X) di akhir proses berarti identik dan Ekuivalen.

Aplikasi Puncak: Ekuivalensi & Minimisasi DFA

Membangun Struktur Komputasi Paling Optimal dari Basis Algoritma

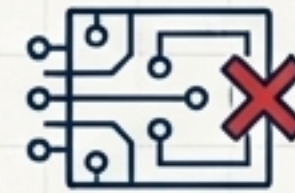
Menguji Ekuivalensi Dua Bahasa ($L = M$)

1. Gabungkan kedua buah DFA ke dalam satu arena matriks Table-Filling.
2. Eksekusi algoritma hingga selesai.
3. Pantau kotak irisan persimpangan pertemuan dari KEDUA Start State asal.



Kesimpulan Mutlak: Jika kotak tersebut TIDAK disilang, maka kedua bahasa membaca himpunan yang persis sama.

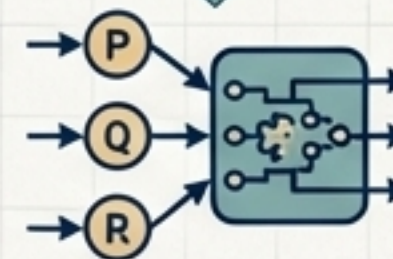
Minimisasi DFA (Reduksi Optimal)



Tahap A: Hapus semua state yang tak terjangkau (*unreachable*) dari sirkuit.



Tahap B: Eksekusi Table-Filling untuk mencari state ekuivalen (kotak kosong).



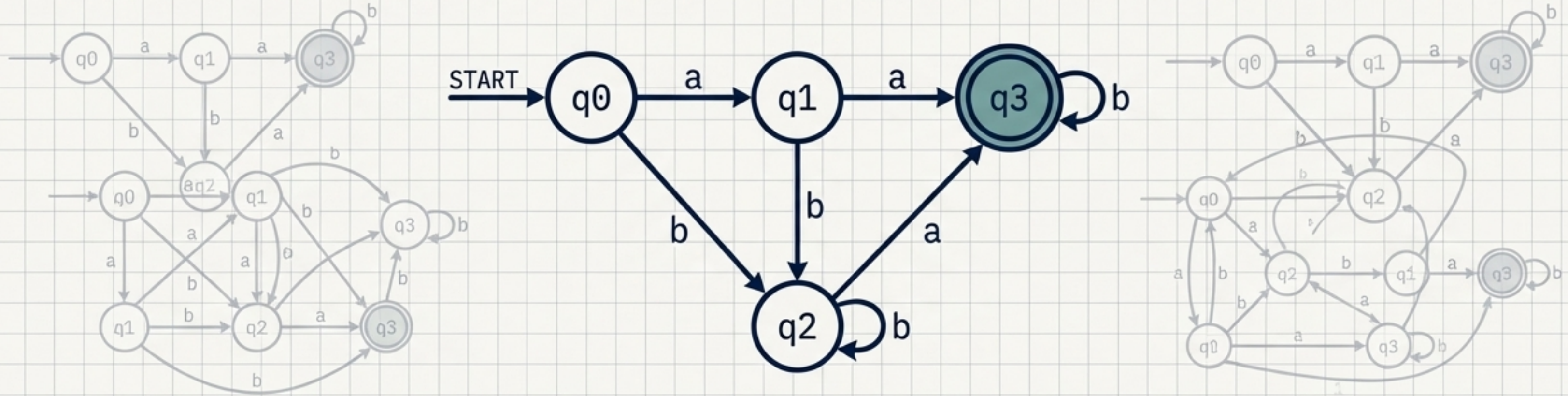
Tahap C: Leburkan blok state yang ekuivalen menjadi satu entitas tunggal yang padat (Fusi).



WARNING: Prosedur minimisasi fusi ini **HANYA** beroperasi valid untuk arsitektur DFA. Menggabungkan state ekuivalen di NFA dapat merusak recognizer komputasi secara permanen!

Garis Akhir: Arsitektur DFA Paling Sempurna

Mengapa Mesin Automata Minimal Tidak Akan Pernah Terkalahkan?



- **Optimalisasi Absolut:** DFA minimal dari hasil Table-Filling Algorithm adalah wujud arsitektur memori paling hemat yang mungkin dirakit secara hukum fisika bahasa reguler tersebut.
- **Keunikan Matematis:** Jika ada mesin DFA lain yang mengklaim lebih kecil atau setara namun bentuknya berbeda, matematika membuktikan bahwa klaim itu mustahil. Struktur ini adalah absolut dan Unik (Isomorfik).
- **Akhir Peta Jalan:** Kita bermula dari mengekspos batasan memori (Pumping Lemma), merakit fungsi baru (Closure), mengukur biaya operasi (Decision), dan tiba pada mahakarya mesin paling efisien (Minimization).